

Universidad Tecnológica de Bolívar

Informe de práctica

Presentado por:

Ana María Rojas Mendoza

Código estudiantil:

T00082483

Curso: Arquitectura de Software

NRC: 1508

Docente: Luis Giovanny Carreno Ortiz

Cartagena, Bolívar

Octubre de 2026

Introducción

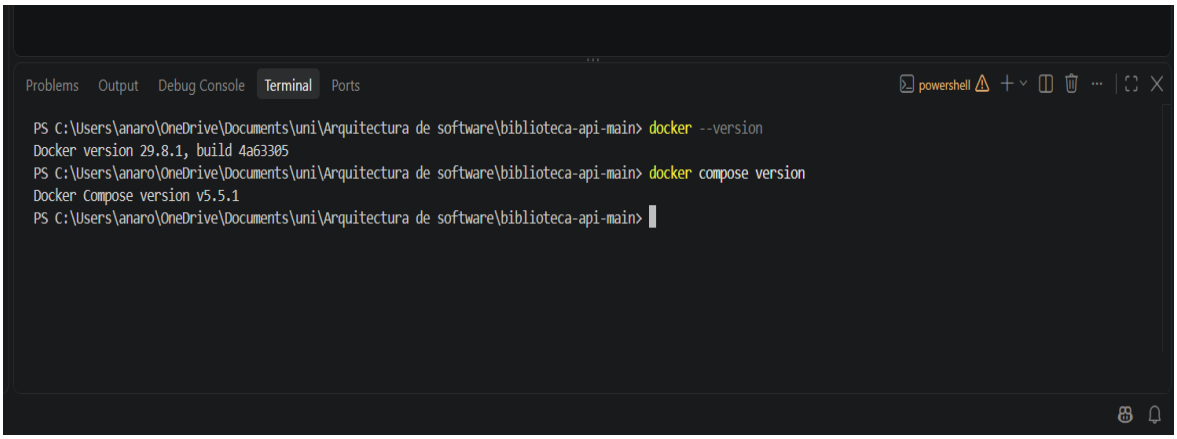
En esta práctica se puso en funcionamiento el proyecto *biblioteca-api*, creado con Go y MySQL, utilizando Docker y Docker Compose. La idea era seguir las instrucciones para instalar y ejecutar la aplicación, comprobar que los contenedores funcionaran correctamente y verificar la conexión entre la API y la base de datos. También se hicieron pruebas para registrar usuarios, iniciar sesión y consultar los libros disponibles.

Fase 1. Verificación e instalación de Docker

Antes de ejecutar el proyecto se verificó que Docker y Docker Compose estuvieran correctamente instalados en el sistema mediante los siguientes comandos:

docker --version
docker compose versión

Los resultados confirmaron que ambas herramientas se encontraban disponibles y listas para utilizarse.

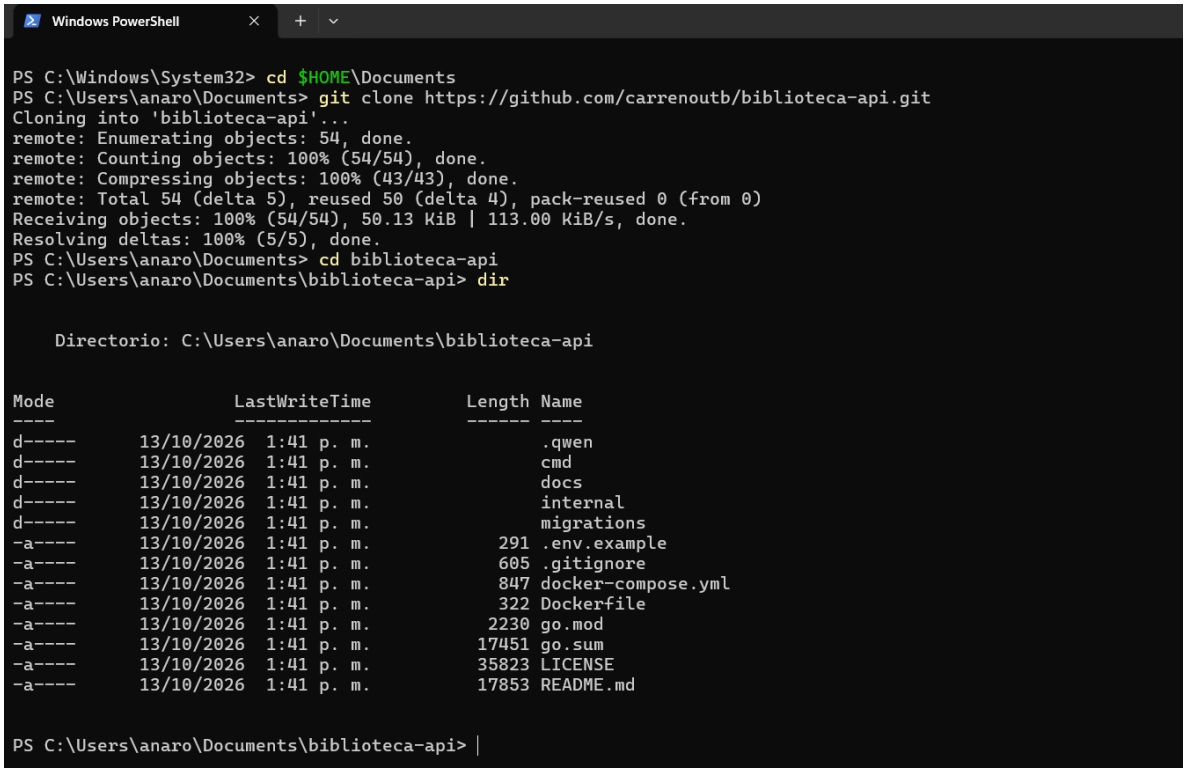


Fase 2. Clonación del repositorio

Se descargó el proyecto desde GitHub utilizando Git.

git clone https://github.com/carrenoutb/biblioteca-api.git
cd biblioteca-api

Posteriormente se verificaron los archivos principales del proyecto.



Fase 3. Análisis de los componentes del proyecto

Antes de ejecutar la aplicación se revisaron los archivos principales:

Dockerfile: Contiene las instrucciones necesarias para construir la imagen Docker de la aplicación desarrollada en Go.
docker-compose.yml: Define los servicios que compondrán el sistema, incluyendo la API y la base de datos MySQL, así como sus puertos, redes y variables de entorno.
.env.example: Archivo plantilla que contiene las variables de configuración requeridas por la aplicación.
migrations/: Contiene los scripts SQL responsables de crear las tablas y registrar datos iniciales en la base de datos.

Fase 4. Configuración del entorno

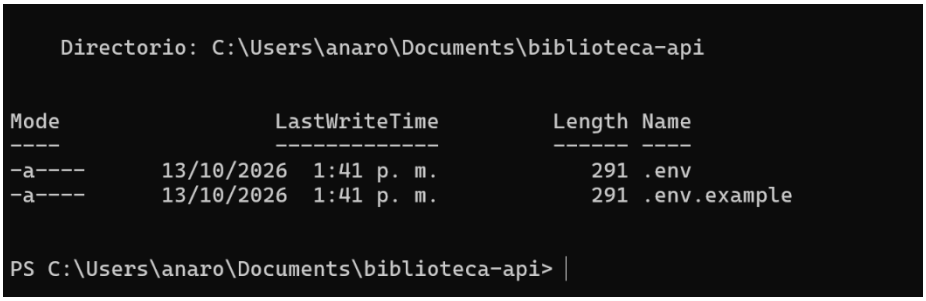
Se creó el archivo .env a partir del archivo de ejemplo.

Copy-Item .env.example .env

Posteriormente se verificó su creación.

Explicación de las variables

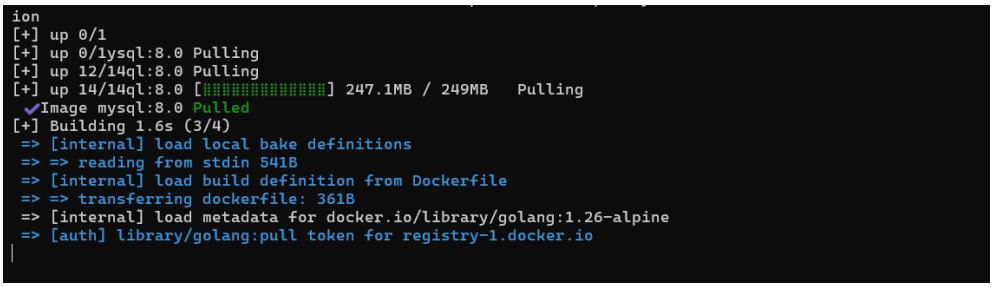
Variable	Descripción
DB_HOST	Nombre del servicio donde se encuentra MySQL.
DB_PORT	Puerto utilizado por MySQL.
DB_USER	Usuario utilizado para conectarse a la base de datos.
DB_PASSWORD	Contraseña del usuario MySQL.
DB_NAME	Nombre de la base de datos utilizada por la aplicación.
JWT_SECRET	Clave utilizada para generar y validar tokens JWT.
JWT_EXPIRATION_HOURS	Tiempo de validez del token en horas.
APP_PORT	Puerto donde se publica la API.

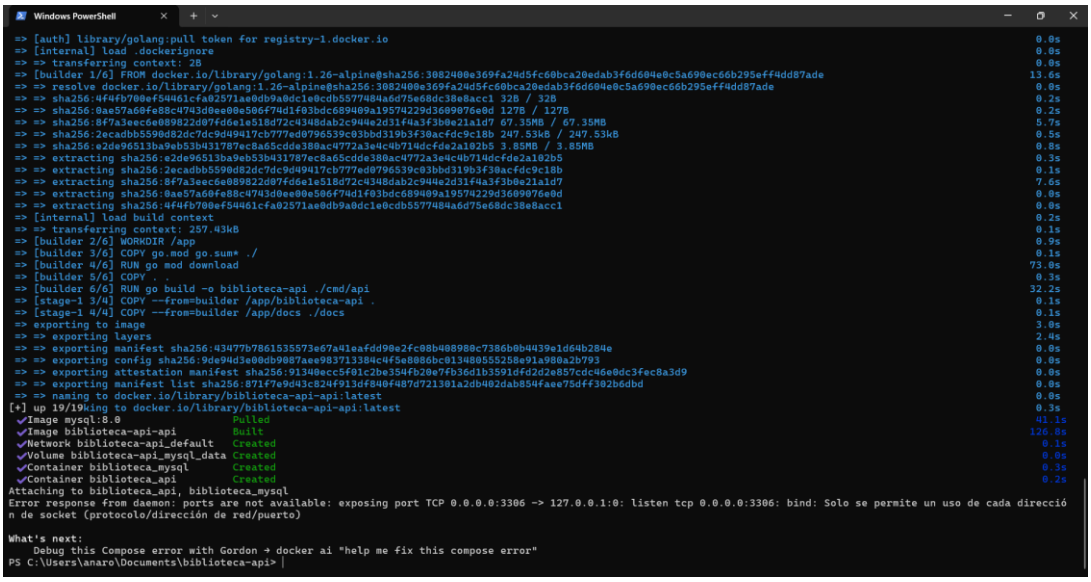


Fase 5. Construcción y ejecución de los contenedores

Una vez configurado el entorno, se inició la aplicación mediante:
docker compose up --build

Este comando realizó las siguientes tareas:
Descargó las imágenes necesarias.
Construyó la imagen de la API.
Inició el contenedor de MySQL.
Ejecutó las migraciones.
Inició la aplicación.



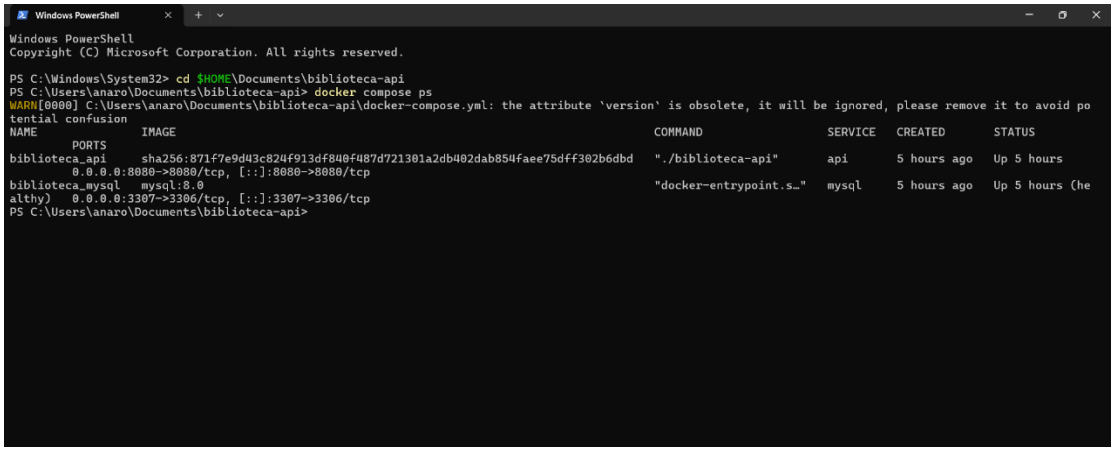


Fase 6. Verificación de los servicios

Para comprobar que los contenedores se encontraban funcionando correctamente se ejecutó:

docker compose ps

Se verificó que:
El contenedor de la API estuviera activo.
El contenedor MySQL mostrara estado healthy.

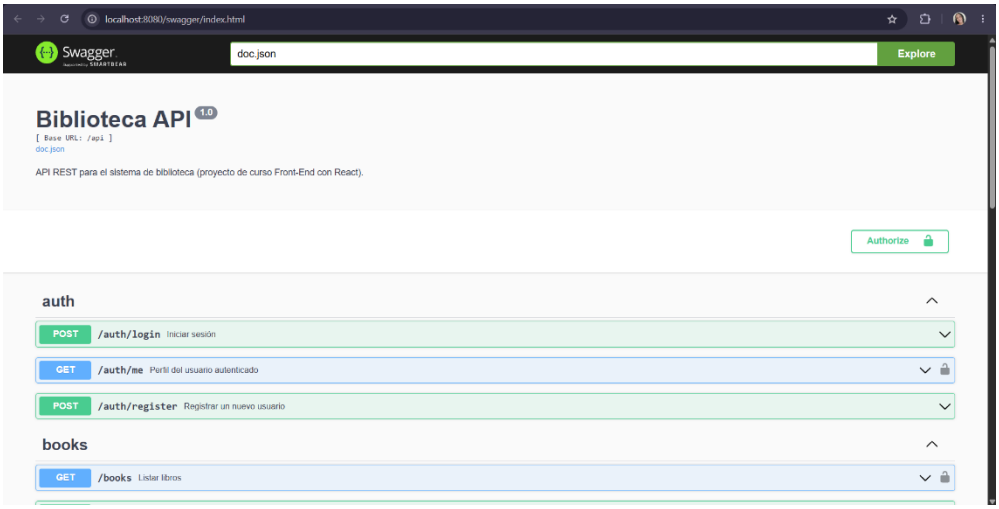


Fase 7. Verificación mediante Swagger

La documentación interactiva de la API fue accesible mediante:

http://localhost:8080/swagger/index.html

Desde esta interfaz fue posible visualizar los endpoints disponibles y conocer su funcionamiento.

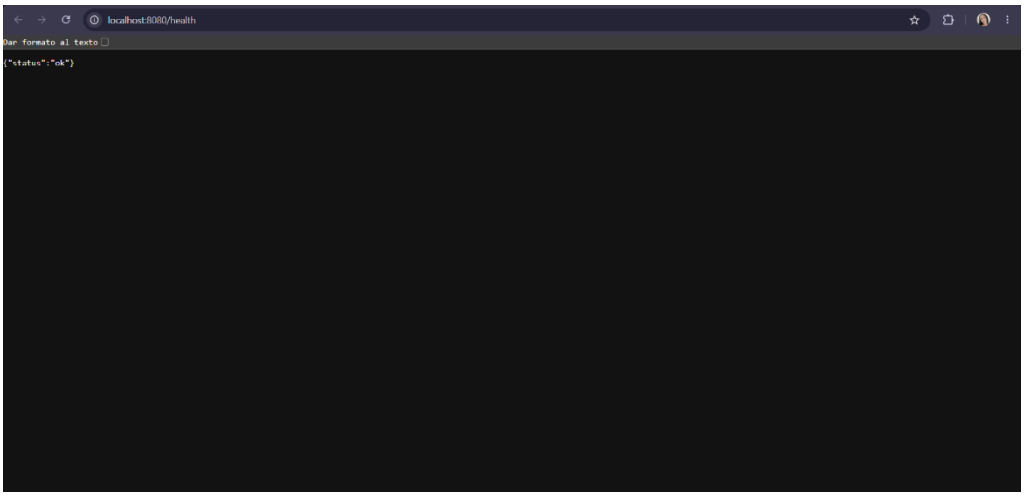


Fase 8. Verificación del estado de la API

Para comprobar la disponibilidad de la aplicación se consultó el endpoint health

http://localhost:8080/health

La respuesta obtenida confirmó que la API se encontraba operativa.

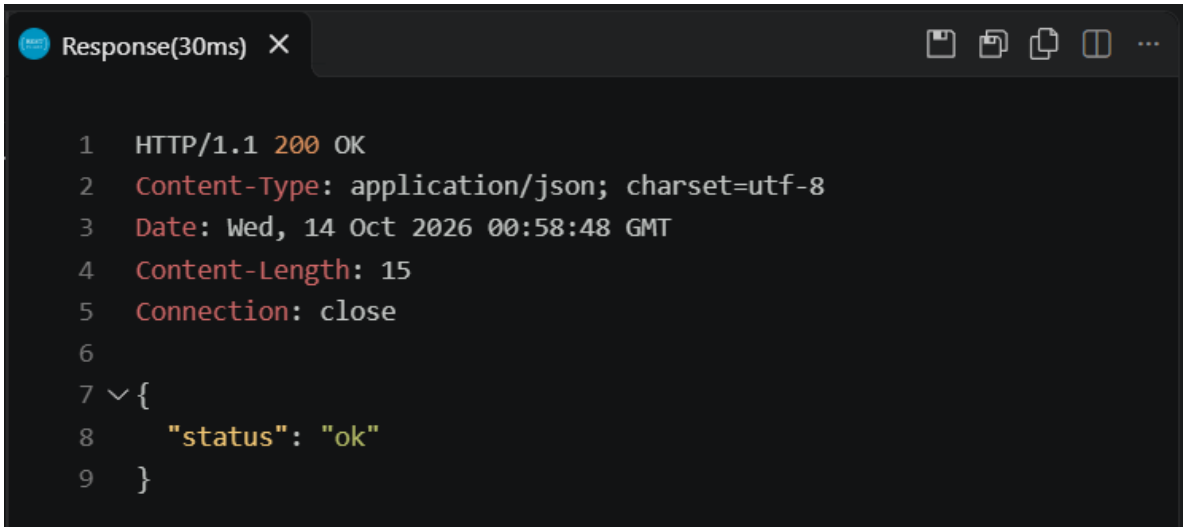


Fase 9. Pruebas funcionales

Se utilizó la extensión REST Client de Visual Studio Code para validar las funcionalidades básicas del sistema.

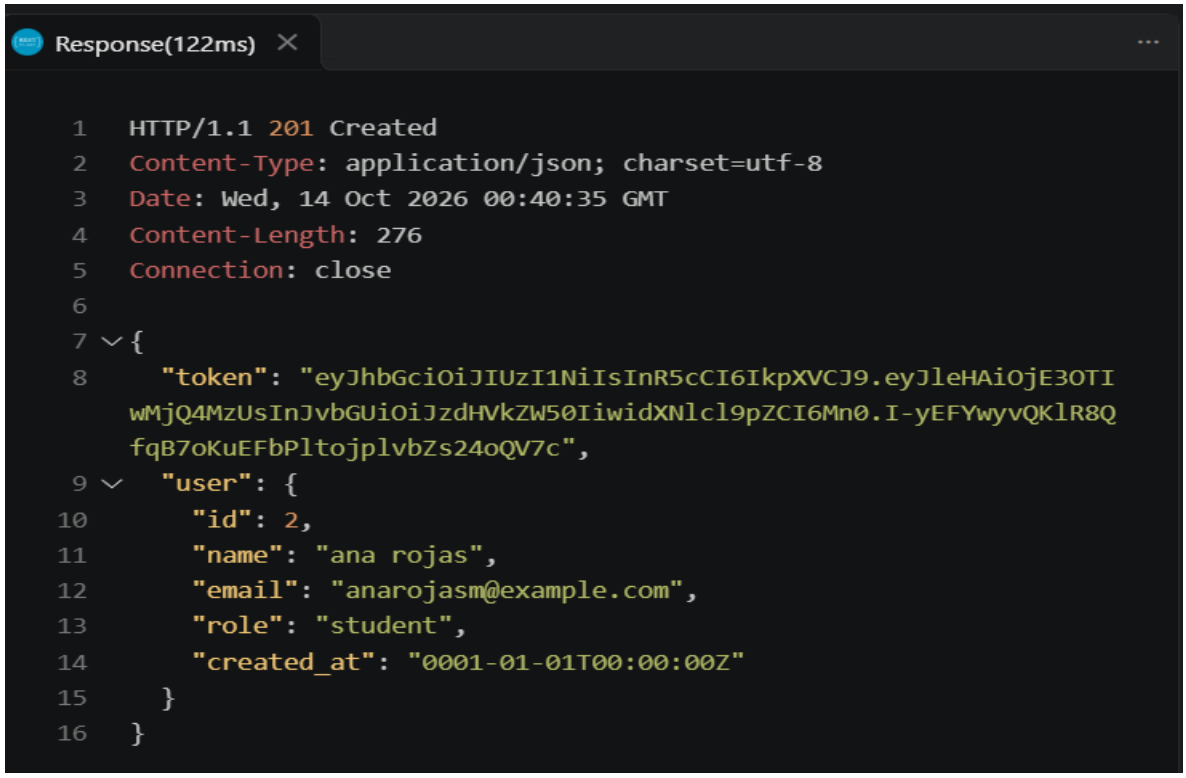
Prueba 1: Estado de la API

Se ejecutó una petición GET al endpoint /health.

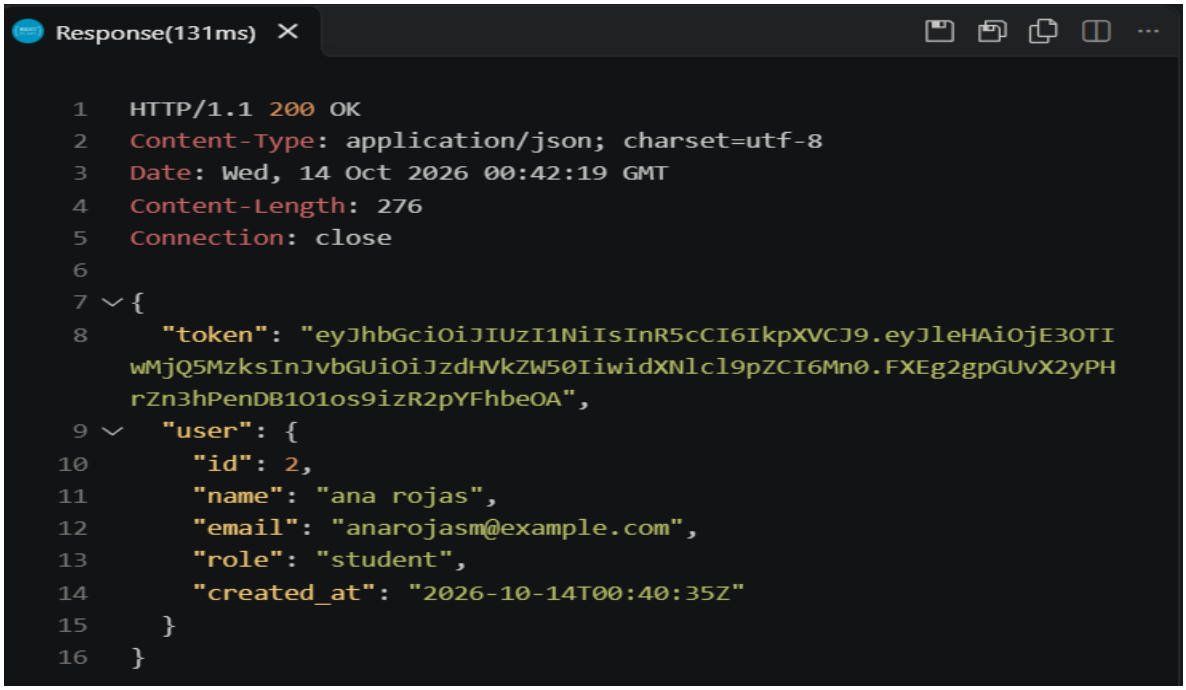


Prueba 2: Registro de usuario

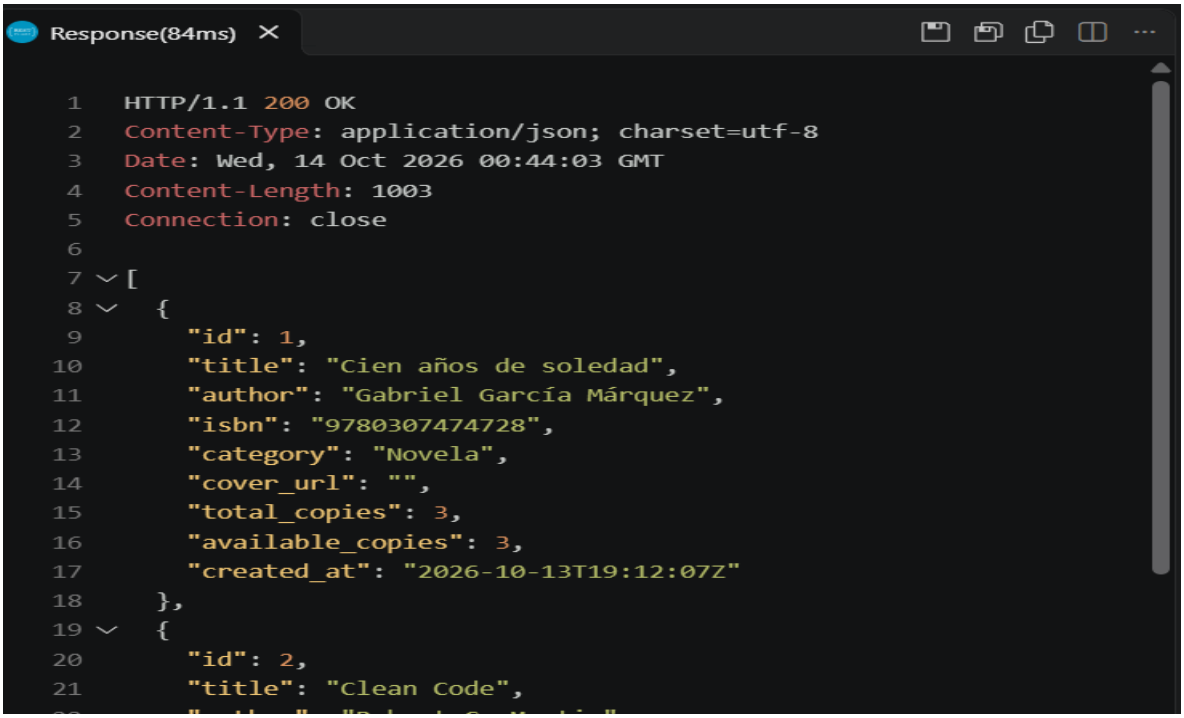
Se registró un nuevo usuario mediante el endpoint correspondiente.



Prueba 3: Inicio de sesión
Se realizó autenticación utilizando las credenciales registradas.



Prueba 4: Consulta de libros
Utilizando el token JWT obtenido durante el inicio de sesión se consultó el listado de libros disponible.



Preguntas de comprensión

1. **¿Qué diferencia hay entre una imagen y un contenedor? Usa un ejemplo de este proyecto.**

Una imagen es una plantilla que contiene todo lo necesario para ejecutar una aplicación, incluyendo código, dependencias y configuración. Un contenedor es una instancia en ejecución de una imagen. En este proyecto, la imagen de biblioteca-api se construye a partir del Dockerfile mediante docker build, mientras que el contenedor se crea cuando se ejecuta docker compose up, permitiendo que la API funcione realmente.

2. ¿Qué hace docker compose que no harías con docker run por separado?

Docker Compose permite administrar múltiples contenedores mediante un único archivo de configuración. En este proyecto inicia la API y MySQL simultáneamente, configura las redes, variables de entorno y dependencias entre servicios. Realizar lo mismo con docker run requeriría ejecutar varios comandos manualmente y configurar cada parámetro por separado.

3. ¿Por qué la API encuentra la base de datos con DB_HOST=mysql y no con 127.0.0.1?

Porque Docker Compose crea una red interna donde cada servicio puede comunicarse utilizando el nombre definido en el archivo docker-compose.yml. En este caso, el servicio de base de datos se llama mysql, por lo que la API puede localizarlo utilizando ese nombre. La dirección 127.0.0.1 apuntaría al propio contenedor y no a la base de datos.

4. ¿Qué borra exactamente docker compose down -v y cuándo lo usarías?

Este comando elimina los contenedores, las redes creadas por Compose y también los volúmenes asociados al proyecto. Se utiliza cuando se desea reiniciar completamente el entorno o eliminar datos persistentes que podrían estar causando problemas.

5. ¿Qué pasaría si cambias DB_PASSWORD en el .env pero no en el docker-compose.yml? ¿Y al revés?

Si las contraseñas son diferentes, la API intentará conectarse utilizando credenciales distintas a las configuradas en MySQL. Como resultado, la conexión a la base de datos fallará y la aplicación no podrá funcionar correctamente. Por esta razón, ambas configuraciones deben mantenerse sincronizadas.

Problemas encontrados y soluciones

Problema 1: Docker Desktop no estaba iniciado

- Síntoma: al ejecutar comandos Docker aparecía un error indicando que no era posible conectarse al daemon.
- Solución: iniciar Docker Desktop y esperar a que el motor Docker estuviera completamente operativo.

Problema 2: Puerto 8080 ocupado

- Síntoma: la API no podía iniciar porque el puerto ya estaba siendo utilizado por otra aplicación.
- Solución: detener la aplicación que ocupaba el puerto o modificar el puerto configurado para la API.

Conclusión

Con esta práctica pude entender mejor cómo funcionan Docker y Docker Compose en un proyecto real de Arquitectura de Software. También aprendí cómo una aplicación puede ejecutarse con todo lo que necesita sin depender de la configuración de cada computador. Además, pude comprobar cómo la API y la base de datos trabajan por separado mediante contenedores. Esto me permitió conocer las ventajas de Docker, como facilitar la instalación, mantener los servicios separados y ejecutar el proyecto en diferentes entornos. Por eso, considero que es una herramienta muy útil para desarrollar aplicaciones modernas.